



VotreID_IA_TP03

November 24, 2023

1 TP02 IA : Le perceptron

1.1 Objectifs du TP

1. Implementer l'algorithme d'apprentissage d'un perceptron
2. Savoir initialiser, importer et préparer les données.
3. Visualiser les résultats
4. Implementer l'algorithme de la descente du gradient

1.2 Définitions et notations

Avant d'aller plus loin dans nos algorithmes pour l'apprentissage des points de notre perceptron, commençons par fixer les notations.

- Nous allons nommer les deux classes positive et negative par, respectivement '1' et '-1'.
- Nous définissons une fonction d'activation $g(\mathbf{z})$ qui recoie comme entrée une combinaison linéaire des valeurs d'entrée $\mathbf{x}$ et les poids $\mathbf{w}$: $(\mathbf{z} = w_1x_1 + \dots + w_mx_m)$
- Si $g(\mathbf{z})$ est plus grand que θ la prediction est 1, sinon -1;
- la fonction g est la fonction seuil simple vu dans le cours: "Heaviside step function"

$$g(z) = \begin{cases} 1 & \text{si } z \geq \theta \\ -1 & \text{sinon.} \end{cases}$$

où

$$z = w_1x_1 + \dots + w_dx_d = \sum_{j=1}^d x_jw_j = \mathbf{w}^T \mathbf{x}$$

$\mathbf{w}$ est le vecteur des poids, et le vecteur $\mathbf{x}$ de dimension d

$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$$

Nous rappelons que si nous posons $w_0 = -\theta$ et $x_0 = 1$



$$g(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ -1 & \text{sinon.} \end{cases}$$

et

$$z = w_0x_0 + w_1x_1 + \dots + w_dx_d = \sum_{j=0}^d x_jw_j = \mathbf{w}^T \mathbf{x}.$$

où

$$z = w_1x_1 + \dots + w_mx_m = \sum_{j=1}^m x_jw_j = \mathbf{w}^T \mathbf{x}$$

1.3 Règle d'apprentissage d'un Perceptron

La règle d'apprentissage du perceptron de Rosenblatt est simple et nous pouvons le resumer comme suit:

1. Itisialiser les poids à 0 ou à des petits nombres tirées aléatoirement.
2. Pour chaque exemple $\mathbf{x}^{(i)}$ de l'ensemble d'apprentissage :
 - calculer la prediction du modele pour $\mathbf{x}^{(i)}$
 - mettre à jour les poids

La mise à jour des poids peut etre écrit comme : $w_j := w_j + \Delta w_j$.

La valeur de la mise à jour à chaque itératon est calculée par la règle d'apprentissage :

$$\Delta w_j = \eta (y_{cible}^{(i)} - y_{predict}^{(i)}) x_j^{(i)}$$

où η est le taux d'apprentissage (une constante entre 0.0 et 1.0).

Rappelons que l'agorithme est in line, tous les poids dans le vecteur $\mathbf{w}$ sont mis à jour simultene-ment pour chaque exemple $\mathbf{x}^{(i)}$.

Pour $d = 2$, nous avons :

$$\Delta w_0 = \eta (y_{cible}^{(i)} - y_{predict}^{(i)})$$

$$\Delta w_1 = \eta (y_{cible}^{(i)} - y_{predict}^{(i)}) x_1^{(i)}$$

$$\Delta w_2 = \eta (y_{cible}^{(i)} - y_{predict}^{(i)}) x_2^{(i)}$$

Rappelons egaleme-nt que dans les deux cas ou la prediction est correcte, notre algo ne réalise pas des mises à jours des poids:

- $\Delta w_j = \eta (-1^{(i)} - -1^{(i)}) x_j^{(i)} = 0$
- $\Delta w_j = \eta (1^{(i)} - 1^{(i)}) x_j^{(i)} = 0$

Cependant, pour les deux cas d'erreur discutés dans le cours les poids sont augmentés dans la bonne direction vers la classe cible.



$$\Delta w_j = \eta(1^{(i)} - -1^{(i)}) x_j^{(i)} = \eta(2) x_j^{(i)}$$
$$\Delta w_j = \eta(-1^{(i)} - 1^{(i)}) x_j^{(i)} = \eta(-2) x_j^{(i)}$$

1.4 Implémentation de l'algorithme

1.4.1 Initialisations

Avant de commencer notre algorithme, voici comment vous pouvez initialiser le vecteur de poids.

Le module random de numpy permet la génération des nombres aléatoires.

Les réels : La fonction `np.random.random(size=None)`

génère des nombres réels (float) aléatoirement, le paramètre size et le nombre des nombres générés (un entier). Les valeurs sont entre 0 et 1, 1 exclus. Exemple:

```
[213]: import numpy as np
np.random.seed(10)
s=np.random.random(5)
print("Les nombres aléatoires générés sont:")
print(s)
```

Les nombres aléatoires générés sont:

[0.77132064 0.02075195 0.63364823 0.74880388 0.49850701]

1.4.2 Exercice :

1. Utiliser la fonction `random.uniform(start, stop)` (voir la documentation) pour générer et afficher un tableau de 10 nombres réels compris entre 10.0 et 20.0
2. Initialiser le vecteur des poids **w** de trois manières différentes.

Le résultat obtenu est un tableau de 5 nombres réels pour une valeur de seed=10.

La fonction seed `np.random.seed` est une fonction de la bibliothèque NumPy qui permet d'initialiser le générateur. En spécifiant une valeur initiale, la fonction s'assure que la séquence des nombres aléatoires générés reste la même sur plusieurs exécutions, fournissant ainsi, un comportement déterministe et permettant la reproductibilité dans la génération de nombres aléatoires. (important pour le debugage)

Les entiers : Syntaxe : `np.random.randint(low, high=None, size=None, dtype=int)`

```
[214]: np.random.seed(10)
s= np.random.randint(1,5,10)
print("Les nombres aléatoires générés sont:")
print(s)
```

Les nombres aléatoires générés sont:

[2 2 1 4 1 2 4 1 2 2]

Le code ci-dessus génère un tableau de 10 entiers aléatoires entre 1 et 5 pour une valeur seed = 10.



```
[242]: import numpy as np
import random
np.random.seed(10)
s = [random.gauss(0, 1) for i in range(10)]
print("Les nombres aléatoires générés sont:")
print(s)
```

Les nombres aléatoires générés sont:

```
[-0.09467376857643092, 0.5172454359615077, -0.9237750020919504,
-0.653911937445536, -0.02150615030349409, 0.9876695825362952,
0.34893742098063946, 0.47928694907235175, -0.007368365880818716,
1.0256903418173837]
```

1.4.3 La classe Perceptron

Nous allons regrouper toutes portions du code vues en cours dans une seule classe.

[242]:

```
import numpy as np

class Perceptron(object):

    def __init__(self, eta=0.01, epochs=50):
        self.eta = eta
        self.epochs = epochs
        self.random_seed = 3436

    def init_params(self, X):
        #self.w_ = np.zeros(1 + X.shape[1])
        self.w_ = np.random.random(1 + X.shape[1])

    def train(self, X, y):
        self.erreurs_ = []
        self.init_params(X)

        for _ in range(self.epochs):
            erreurs = 0
            for xi, target in zip(X, y):
                update = self.eta * (target - self.predict(xi))
                self.w_[1:] += update * xi
                self.w_[0] += update
                erreurs += int(update != 0.0)
            self.erreurs_.append(erreurs)
        return self

    # C'est notre fonction g(x)
    # Le paramètre x est un exemple
    def potentiel(self, X):
```



```

return np.dot(self.w_[1:], X) + self.w_[0]

def predict(self, X):
    return np.where(self.potentiel(X) >= 0.0, 1, -1)

```

1.4.4 Exemple 1

Préparation de l'ensemble d'entraînement (Manuellement) Repréons l'exemple, vu en cours, de la fonction OU logique $d = 4$, on peut saisir les données manuellement en créant le vecteur X :

$$\mathbf{X} = \begin{bmatrix} x^1 \\ x^2 \\ x^3 \\ x^4 \end{bmatrix} = \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \\ x_1^3 & x_2^3 \\ x_1^4 & x_2^4 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

```

[243]: d = 2
n = 4
#une manière
X1 = np.array([0, 0, 1, 1])
X2 = np.array([0, 1, 0, 1])
X = np.vstack([X1, X2])
y = np.array([0, 1, 1, 1])
# Une autre manière
X = np.array([[0,0], [0,1], [1,0], [1,1]])

```

Entraîner le modèle On peut maintenant entraîner et exploiter notre modèle.

```

[258]: %matplotlib inline
import matplotlib.pyplot as plt

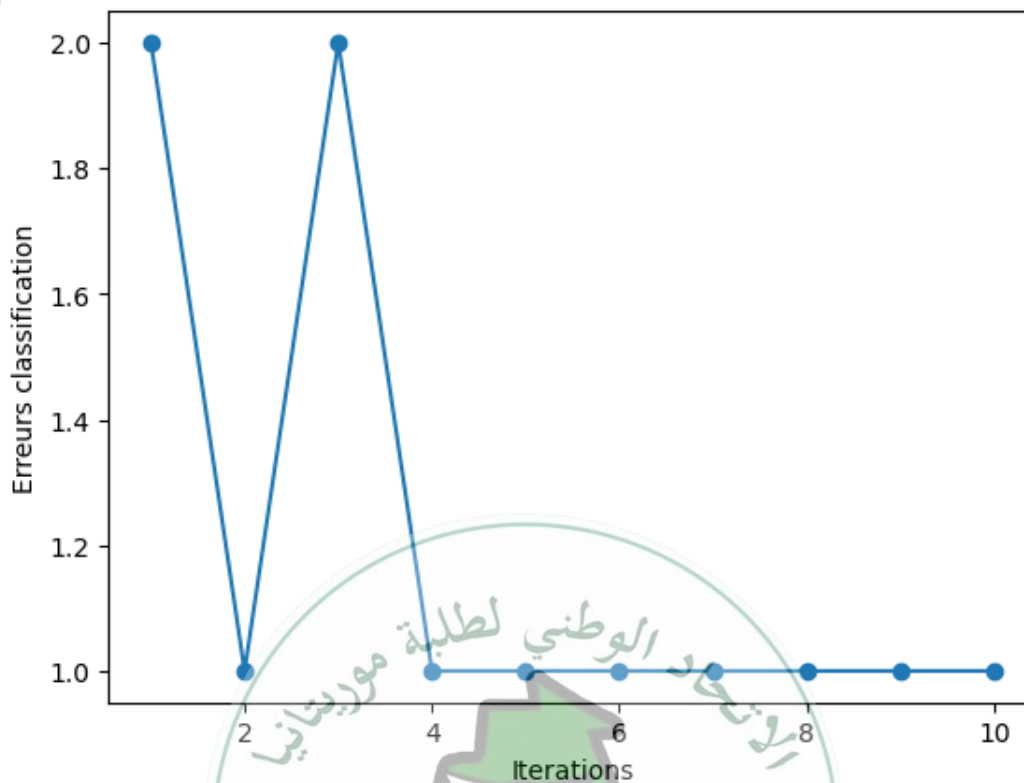
ppn = Perceptron(epochs=10, eta=0.8)

ppn.train(X, y)
print('Weights: %s' % ppn.w_)

plt.plot(range(1, len(ppn.erreurs_)+1), ppn.erreurs_, marker='o')
plt.xlabel('Iterations')
plt.ylabel('Erreurs classification')
plt.show()

```

Weights: [0.07961309 1.90545992 1.93071931]



1.4.5 Exemple 2

Préparation de l'ensemble d'entraînement (Automatiquement) En pratique, les données sont rarement saisies manuellement. Elles sont souvent importées à partir d'une base de données ou générées. L'option d'importation est la plus utilisée.

Voyons maintenant comment importer le célèbre jeu de données IRIS.

La base de données IRIS C'est une base de données regroupant les caractéristiques de trois espèces de fleurs d'Iris, à savoir Setosa, Versicolour et Virginica. Chaque ligne de ce jeu de données est une observation des caractéristiques d'une fleur d'Iris. Ce dataset décrit les espèces d'Iris par quatre propriétés : longueur et largeur de sépales ainsi que longueur et largeur de pétales (en cm). La base de données comporte 150 observations (50 observations par espèce).

L'objectif est de prédire la classe (la variété) à partir des quatre variables explicatives (longueurs et largeurs des sépales et des pétales).

NB: Noter que cette base de données est disponible automatiquement dans certaines bibliothèques de machine learning comme Scikit-learn ou Pandas. Dans ce cas, vous pouvez charger directement la base de données et l'utiliser.

[231]: `import pandas as pd`



```
df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data', header=None)

# setosa and versicolor
y = df.iloc[0:100, 4].values

y = np.where(y == 'Iris-setosa', -1, 1)

# sepal length and petal length
X = df.iloc[0:100, [0,2]].values

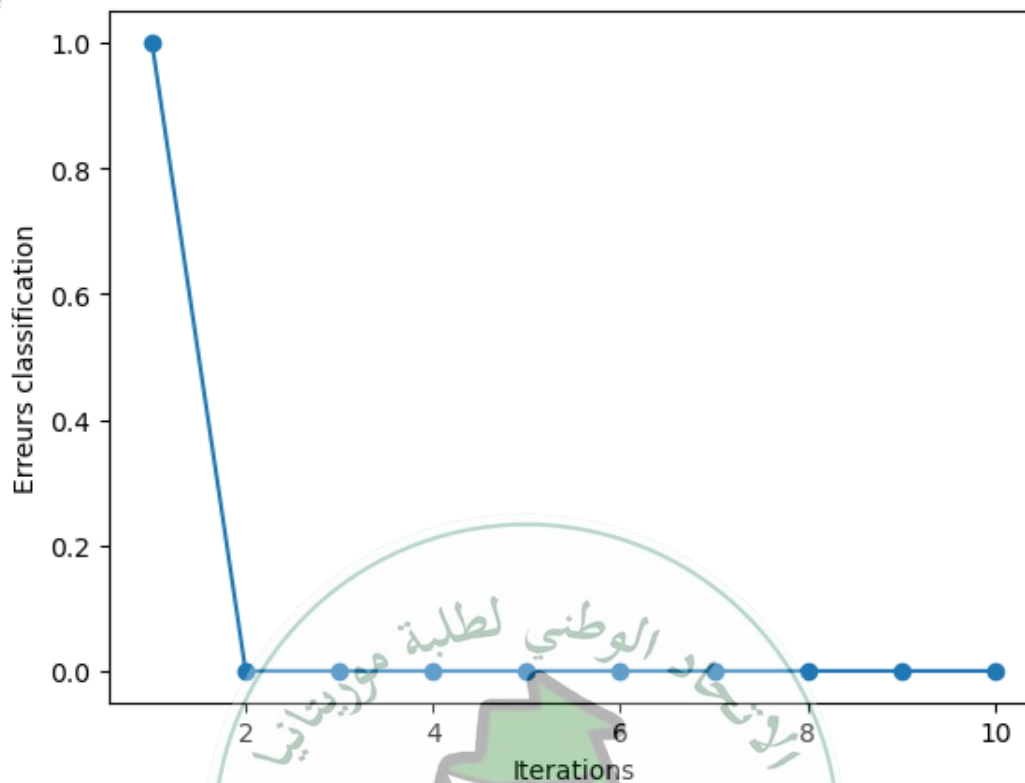
%matplotlib inline
import matplotlib.pyplot as plt

ppn = Perceptron(epochs=10, eta=0.1)

ppn.train(X, y)
print('Weights: %s' % ppn.w_)

plt.plot(range(1, len(ppn.errors_)+1), ppn.errors_, marker='o')
plt.xlabel('Iterations')
plt.ylabel('Erreurs classification')
plt.show()

1== [0. 0. 0.]
2== [0.08833981 0.68535982 0.95339335]
Weights: [-0.11166019 -0.33464018  0.67339335]
```



La deuxième option est de télécharger le fichier de données directement à partir du site (Fichier en format CSV) Pour cela, nous pouvons utiliser la commande wget. Vous pouvez également télécharger manuellement le fichier à partir de votre navigateur dans le dossier courant.

```
[221]: %%bash
wget https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data
```

```
--2023-11-22 20:23:17-- https://archive.ics.uci.edu/ml/machine-learning-
databases/iris/iris.data
Résolution de archive.ics.uci.edu (archive.ics.uci.edu)... 128.195.10.252
Connexion à archive.ics.uci.edu (archive.ics.uci.edu)|128.195.10.252|:443...
connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : non indiqué
Sauvegarde en : « iris.data.2 »
```

OK ...

21.9M=0s

2023-11-22 20:23:19 (21.9 MB/s) - « iris.data.2 » sauvegardé [4551]



En suite nous allons lire les données dans un ndarray. Noter que la fonction loadtxt ouvre le fichier et charge les données, donc pas besoin d'ouvrir le fichier en lecture.

```
[259]: # valeurs séparées par virgule, la dernière colonne étant une chaîne de
        ↪ caractère nous sommes obligés de spécifier l
        # le type de chaque colonne
        # Skiprow = 0 saute la première ligne
        # S15 type str sur 15 bytes
        # Aller à la documentation pour avoir plus d'options

data = np.loadtxt("iris.data",
                  dtype={'names': ('sepal length', 'sepal width', 'petal length', 'petal_
        ↪ width', 'label'),
                          'formats': (float, float, float, float, '|S15')}},
                  delimiter=',', skiprows=0)
```

- Afficher data pour voir le contenu de la base de données
- Constater que les données sont regroupées en fonction du type de la fleur (dernière colonne) en groupe de 50

```
[223]: # Je peux récupérer la dernière colonne à part. la numérotation commence de 0.
labels = np.loadtxt("iris.data", delimiter=',', usecols=[4], dtype="str",
        ↪ skiprows=0)
# Et les features à part
features = np.loadtxt("iris.data", delimiter=',', usecols=[0,1,2,3], skiprows=0)
```

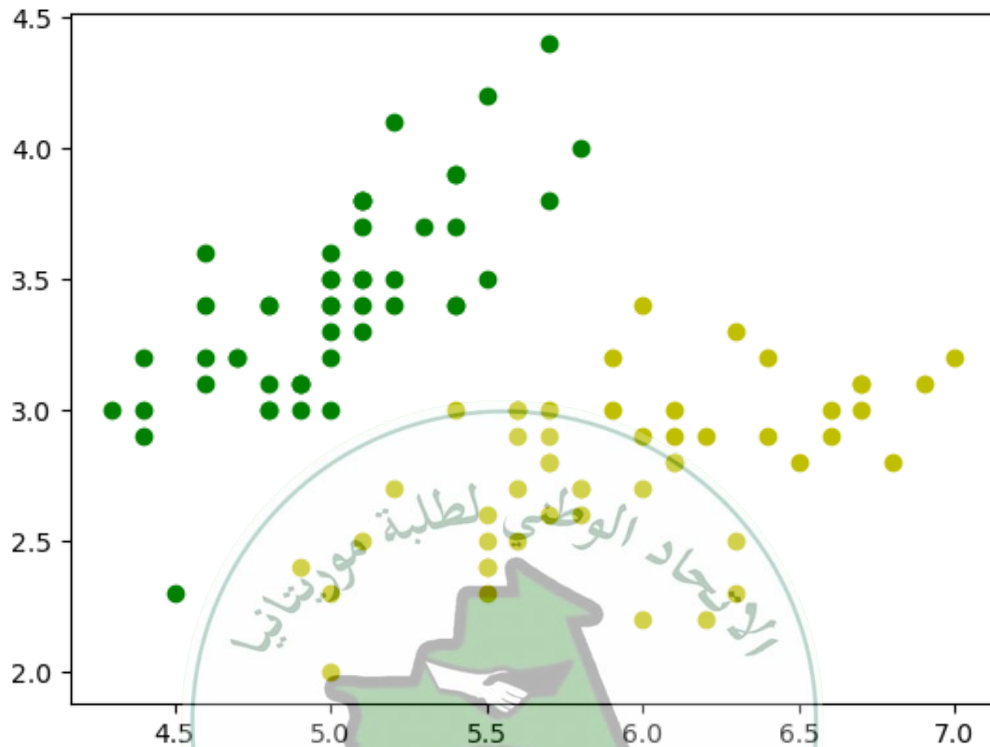
Je vais donc extraire les données de deux premières classes : 'Iris-setosa' et 'Iris-versicolor'

```
[224]: # Récupérer les 100 premières lignes pour les 4 colonnes
#X = features[0:100]
# Récupérer les 100 premières lignes pour les deux premières colonnes
X = features[0:100, [0,1]]

#Je fais la même chose pour les labels (y cibles)
y = labels[0:100]
# conversion des valeurs 'Iris-setosa' et 'Iris-versicolor' en -1 et 1
y = np.where(y == 'Iris-setosa', -1, 1)

#Je trace la longueur et la largeur des fleurs (Première et deuxième colonne)
        ↪ pour les deux classes
# 'Iris-setosa' et 'Iris-versicolor'
# Iris-setosa en vert et l'autre en jaune.
plt.figure()
plt.scatter(X[y == -1][:, 0], X[y == -1][:, 1], color='g', label='-1')
```

```
scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='y', label='1')
plt.show()
```



On remarque que les données de ces deux classes peuvent être linéairement séparées. Un perceptron serait capable de départager les deux classes.

1.4.6 Exercice :

1. Afficher les dimensions du vecteur des caractéristiques (features) X
2. Afficher les dimensions du vecteur étiquettes de classe y
3. Le vecteur y contient les résultats attendus pour les 100 premières observations soit les données de la classe 'Iris-setosa' et de la classe 'Iris-versicolor', créer maintenant un vecteur y pour les deux classes Iris-setosa et Iris-virginica'
4. Extraire les features correspondants dans X
5. Ne conserver que la troisième et quatrième colonne dans X
6. Afficher les données X en donnant à chaque classe une couleur diff.
7. Dites nous si les données sont linéairement séparables ?
8. Peut-on utiliser le modèle de perceptron linéaire pour classer ces données ?
9. Reprendre les questions 4 à 8 pour les deux classes 'Iris-versicolor' et 'Iris-virginica'

Entraînement de notre modèle (perceptron linéaire à seuil)



```
ppn = Perceptron(epochs=1000, eta=0.2)
```

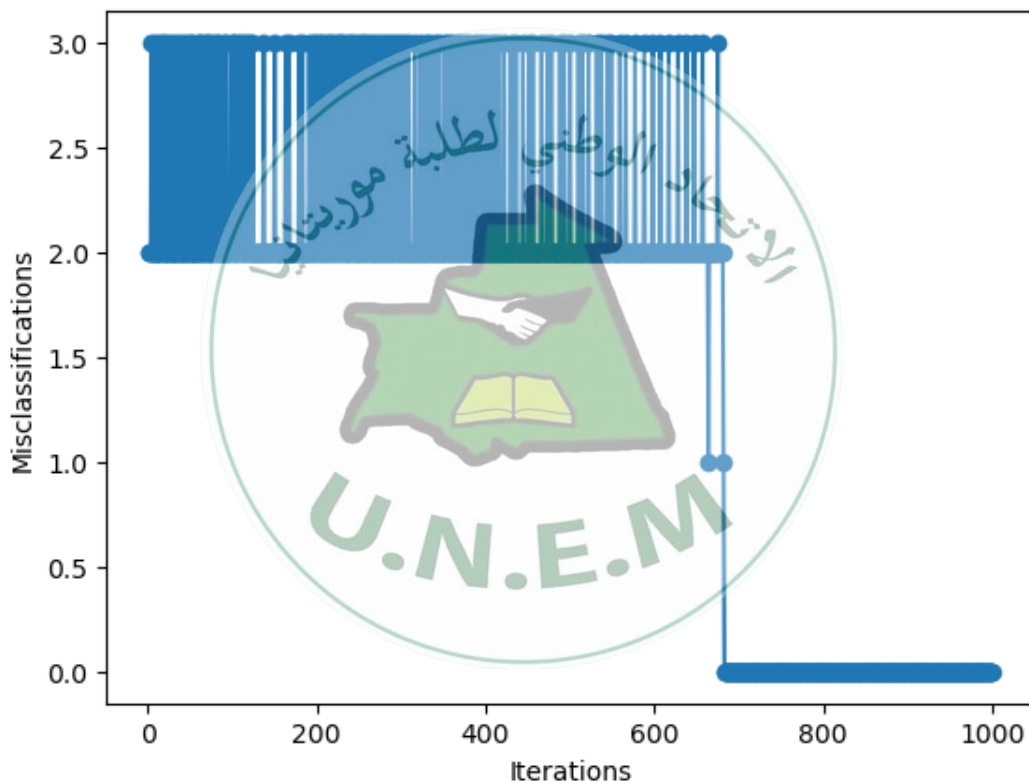
```
ppn.train(X, y)
print('Weights: %s' % ppn.w_)
```

```
1== [0. 0. 0.]
```

```
2== [0.19806286 0.76053071 0.16911084]
```

```
Weights: [-49.00193714 31.40053071 -40.15088916]
```

```
[227]: plt.plot(range(1, len(ppn.errors_)+1), ppn.errors_, marker='o')
plt.xlabel('Iterations')
plt.ylabel('Misclassifications')
plt.show()
```



1.4.7 Exercice :

Executer plusieurs fois et faites varier les paramètres du modèle pour chaque execution. - Poser epochs = 400 et constater que votre modèle n'est pas bon. - Faites la meme chose avec le taux d'apprentissage



Faire une prediction

Notre modèle étant entraîné, nous pouvons maintenant faire des predictions. Etant données les valeurs de “Sepal Length” et “Sepal Width” (nos features : première et deuxième colonne) pour une (ou plusieurs) fleurs, notre objectif est d’interroger notre modèle afin de prédire la classe de cette (ou ces) fleurs d’IRIS.

```
[228]: #prenons le premier exemple de notre ensemble d'entraînement
Xr= np.array([5.1, 3.5])
print(Xr)
res= ppn.predict(Xr)
print(res, '---->', np.where(res==-1, 'Iris-setosa', 'Iris-versicolor'))
```

```
[5.1 3.5]
-1 ----> Iris-setosa
```

```
[229]: #Une valeur quelconque
Xr= np.array([7, 7])

res= ppn.predict(Xr)
print(res, '---->', np.where(res==-1, 'Iris-setosa', 'Iris-versicolor'))
```

```
-1 ----> Iris-setosa
```

1.4.9 Exercice :

- Tester le modèle entraîné avec 5 exemples de l’ensemble d’entraînement
- Regarder le nuage des points des données que nous avons déjà tracé et tester maintenant le modèle entraîné avec des points bien choisis ?
- Écrire l’équation de l’hyperplan (Ici une droite) qui sépare les deux classes.
- Identifier deux points sur la droite séparateur et la tracer

1.4.10 Problème 1 :

En s’inspirant l’exemple ci-dessous et en utilisant la même base de données, nous souhaitons maintenant entraîner notre modèle pour séparer la première classe ‘Iris-setosa’ et la troisième classe ‘Iris-virginica’

I- Préparation des données 1. Charger les données depuis le fichier iris.data 2. Extraire les données features de deux fleurs concernées dans le vecteur X. 3. Ne conserver que la troisième et quatrième colonne dans X 4. Afficher les dimensions du vecteur des caractéristiques (features) X et assurez-vous que votre extraction est correcte 5. Faites la même chose pour le vecteur des étiquettes y 6. Visualiser les données et vérifier la séparabilité de deux classes.

II- Entraînement du modèle 1. Entraîner le modèle sur l’ensemble d’entraînement (X,y) 2. Tracer l’erreur et ajuster vos paramètres

III- Tester le modèle 1. Prendre quelques exemples de l’ensemble d’entraînement et tester le modèle. 2. Tester avec des données choisies aléatoirement



Problème 2 :

Realiser toutes les parties du probleme 1 mais en remplaceant la classe 'perceptron' par une classe 'perceptronDg' que vous allez écrire où l'apprentissage est fait avec la méthodes de la descente du gradient vu en cours.

